

东软秘密

SDD 工程方法指南

东软集团股份有限公司

目录

- 一、SDD 方法概述 1
 - 1.1 SDD 的核心价值 1
 - 1.2 SDD 全流程概览 1
- 二、按项目类型选择 SDD 策略 2
 - 2.1 策略选择决策树 2
- 三、三种 SDD 开发模式 3
 - 3.1 三种模式总览 3
 - 3.2 模式一：基于架构设计的对话式 AI Coding 4
 - 3.3 模式二：Spec-First 5
 - 3.4 模式三：Spec-As-Source（规范即源码） 7
 - 3.5 工程结构索引要求（所有模式统一） 8
- 四、SDD 落地的组织建议 11
 - 4.1 事业部落地 SDD 的总体思路 11
 - 4.2 组织结构与角色建议 12
 - 4.3 项目特征与模式匹配 12
 - 4.4 落地路线图 13
 - 4.5 模式设定及核验 13
- 五、SDD 全流程的准备工作 14
 - 5.1 团队准备 14
 - 5.2 BA/SA 阶段准备 14
 - 5.3 SDD 编码前准备 15
 - 5.4 项目宪法准备（SA → SDD 的桥梁） 17
- 六、SDD 实施通用原则 17
 - 6.1 五连续统一原则 17
 - 6.2 Change 粒度控制 18
 - 6.3 人工确认是保障 18
 - 6.4 Spec 管理与同步 18
- 七、常见问题与应对策略 18
 - 7.1 AI 幻觉 18
 - 7.2 AI 偷懒 19
 - 7.3 上下文过载/AI 记忆丢失 19
 - 7.4 文档与代码不同步 19
- 八、效能提升关键因素 19
- 九、子文件索引 20

一、SDD 方法概述

规格驱动开发（SDD） 是一种以规格文档（Spec）为核心载体的软件开发方法，通过“先写规格，再写代码”的原则，确保 AI 编码助手在充分理解架构设计和需求约束的前提下才开始实现。SDD 将开发过程拆解为 **Specify → Plan → Tasks → Implement** 四个明确阶段，每个阶段的产出物需经人工确认后才进入下一阶段。

1.1 SDD 的核心价值

价值维度	说明
知识显性化	将散落在个人经验中的业务知识、架构决策转化为结构化、可复用的组织资产
AI 受控执行	Spec 作为“人机契约”，约束 AI 在既定轨道上生成代码，降低幻觉和偷懒风险
质量可追溯	需求→架构→Spec→代码的完整链路可追溯，变更影响范围清晰可控
SDD 工具无关性	方法不依赖单一的 SDD 工具：OpenSpec/SpecKit/BMad Method 等多种 SDD 工具均可落地

1.2 SDD 全流程概览

阶段一：业务架构设计（BA）
| 价值流 → 业务能力 → 信息概念
▼

阶段二：解决方案架构设计（SA）
| 系统范围 → 构建块视图 → 运行视图 → 横切关注点 → 数据架构（DA）
▼

阶段三：SDD 代码生成
| ① 配置 constitution + config
| ② 探索 → 理解架构、明确变更范围
| ③ 提案 → proposal + design + specs + tasks
| ④ 实现 → 逐任务实现代码
▼

阶段四：测试与验证

| 单元测试 → 集成测试 → 系统测试

▼

阶段五：归档

| 同步 delta specs 到主规格 + 移入 archive 目录

注：各阶段的详细操作指南参见 [SDD 工具手册](#)。

二、按项目类型选择 SDD 策略

不同类型的项目应采用不同的 SDD 执行策略：**新开发项目**采用 **Spec 优先**、**存量项目**采用**逆向 Spec**，选择依据是项目的新旧状态与知识沉淀方向，核心原则是**投入要与项目生命周期匹配**：

项目类型	推荐 SDD 策略	适用场景	详细指南
新开发项目	Spec 优先	新项目、核心产品、长期维护项目、平台型应用	新开发项目 SDD 实施指南
存量项目	逆向 Spec	存量系统规范化改造、存量系统维护增强、原型验证、短期外包项目	存量项目 SDD 实施指南

2.1 策略选择决策树

项目是全新开发吗？

|—— 是 → Spec 优先（架构设计 → 规格 → AI 产码）

| └—— 完整 4A 架构设计 + Spec 体系

|—— 否 → 是存量系统

└—— 逆向 Spec（4A 架构设计 → 分模块逆向生成 Spec → 维护增强）

关键原则：Spec 优先策略的核心价值在于“长期复利”——Spec 写一次，多轮迭代持续受益。存量项目采用逆向 Spec 策略，先建立 4A 架构基线，再分模块逆向生成 Spec，保证维护质量。

注：落地 Spec 优先策略时，需根据项目特征进一步选择三种 SDD 开发模式之一（基于架构设计的对话式 AI Coding / Spec-First / 规范即源码），详见 三、三种 SDD 开发模式。

三、三种 SDD 开发模式

SDD 的落地方式按 **SDD 实施深度**（Spec 管理精细度与 AI 委托程度）划分为三种模式，实施深度越高，AI 委托程度越高。三种模式均以“架构设计先行”为前提。

3.1 三种模式总览

SDD 的落地方式按 **SDD 实施深度**（Spec 管理精细度与 AI 委托程度）划分为三种模式：

模式	定义与特点	适用定位	Spec 粒度	AI 一次性产码可用率
模式一：基于架构设计的对话式 AI Coding	架构师编写架构设计文档（含核心功能/非功能详细设计与页面原型），开发人员通过对话方式 AI Coding，产码完成后 强制逆向更新设计文档 ；不使用 SDD 工具	应急/兜底：短期紧急交付、原型验证、Bug 修复、技术迁移/基础设施等快速交付或非业务处理类场景	对话级（设计文档中）	60%~70%
模式二：Spec-First	使用 SDD 工具（OpenSpec/SpecKit/BMad）将架构设计与规范结构化，先写 Spec 再产码，Spec 作为长期资产与代码同步演进	主流主力：新开发、核心产品、平台中台、系统集成、数据等绝大多数常规项目	Change 级（结构化）	80%~90%
模式三：Spec-As-Source（规范即源码）	Spec 精细到方法粒度，规范是主要工件，人只编辑规范，代码完全由 AI 生成并通过测试验证，代码是 Spec 的派生产物	知识驱动型重点项目：核心模块、领域规则密集、高可靠性、长期演进的领域	方法级	≈100%

数据来源于 9 个 SDD 试点项目的实践经验，仅供参考。

模式选择说明：

1. 模式一是历史做法，后续项目不推荐：它源自以往系统的开发方式，对员工与 AI 交互能

东软秘密

力要求较高，且需依赖组织管理手段才能保证设计文档与代码一致，风险较大。本次纳入作为存量/紧急场景，以及技术迁移等非传统业务处理类场景的兜底选项。

2. 模式二与模式三均使用 SDD 工具，是规模化落地的推荐模式，二者区别在于 Spec 细化深度与 AI 委托程度。

核心规律: 三种模式均以“架构设计先行”为前提。SDD 实施深度越高 → Spec 越精细 → AI 产码质量越高、功能遗漏率越低，但前期架构与 Spec 编写投入成倍增加。项目应根据质量要求、资源约束与知识密集度选择匹配的模式。

3.2 模式一：基于架构设计的对话式 AI Coding

定义: 架构师编写架构设计文档（BA/AA/DA/TA，并包含核心功能的详细设计与页面原型），开发人员基于该设计文档通过对话方式 AI Coding；产码完成后，开发人员**逆向更新设计文档（强制）**，保持文档与代码一致。

工作方式:

架构师：编写架构设计文档（含核心功能/非功能详细设计 + 页面原型）



开发人员：基于设计文档，对话式 AI Coding（Copilot/Agent）



开发人员：人工逆向更新设计文档，保持文档与代码一致（强制）

SDD 判定边界: AI 产码后**未逆向更新设计文档**的，不属于 SDD。逆向更新设计文档是模式一的硬性要求，也是其知识沉淀的唯一途径。

适用项目特征/场景:

- 短期紧急交付、原型验证、迭代频率极高的项目；
- 存量系统 Bug 修复、紧急热修复等一次性应急变更；
- 客户要求极短交付周期、无暇建立完整 Spec 体系的场景；
- 技术迁移/基础设施变更等非业务处理类场景；
- 仅在模式二/三不可行时作为兜底选项。

Spec 提交要求:

维度	要求
设计文档范围	架构设计文档 + 核心功能/非功能详细设计 + 页面原型等

Spec 粒度	核心模块/功能在设计文档中，仅显性化核心业务规则、接口协议与关键约束
编写顺序	架构师先行编写设计文档 → 对话式 AI Coding → 逆向更新设计文档（强制）
提交时机	代码提交时 必须同步提交逆向更新的设计文档 ；未更新文档的提交不视为 SDD
人工确认	架构师确认设计文档、开发核查 AI 产码并维护文档一致性

不推荐原因（历史做法说明）：

- 对员工能力要求较高：需具备架构设计、AI 交互与代码核查的综合能力，并自行弥补 Spec 粒度不足带来的设计缺口；
- Spec 粒度不统一：容易超出模型记忆上限，导致 AI 产码时丢失需求；
- 文档与代码一致性依赖组织管理手段强制保证，长期维护存在知识断层与 Spec-Code 漂移风险；
- 产码后极易存在功能遗漏率高、“补课”成本高的问题；

模式升级路径（模式一 → 模式二）：

对于因紧急交付等原因采用模式一的项目，待项目交付稳定后，建议通过**逆向 Spec** 方式逐步升级至模式二：

1. 以当前代码与设计文档为基线，逆向生成结构化 Spec（BA/AA/DA/TA + 模块级 Spec）；
2. 后续新功能迭代采用模式二的 Spec-First 流程（Specify → Plan → Tasks → Implement）；
3. 存量模块按优先级分批次补全 Spec，逐步覆盖核心业务链路。

逆向 Spec 的具体操作参见 [存量项目 SDD 实施指南](#)。

3.3 模式二：Spec-First

定义：使用 SDD 工具（OpenSpec/SpecKit/BMad Method）对架构设计与规范进行结构化管理，每次变更先修改 Spec 再修改代码，Spec 作为长期资产与代码同步演进。

工作方式：Specify → Plan → Tasks → Implement 四阶段，每阶段产出物经人工确认后进入下一阶段；每个 Change 完成闭环（含归档与 Delta Spec 同步）。

适用项目特征/场景：

东软秘密

- 新开发项目、核心产品、长期维护项目、平台型应用；
- 需持续迭代、多人协作、多 Change 并行的业务系统；
- 中长期生命周期，对知识可追溯、质量一致性与人员流动韧性有要求的产品；
- 存量项目的新功能迭代（结合逆向 Spec 建立基线后）；
- 系统集成/接口适配；
- 数据/算法密集型项目；
- 组织级 SDD 落地的推荐主流模式，覆盖绝大多数常规项目。

Spec 提交要求：

维度	要求
架构设计文档	4A 架构设计文档（BA/AA/DA/TA）先行完成并随 Spec 一并提交，作为 Spec 的输入基线；架构变更时同步更新文档
Spec 粒度	结构化（Change 级），单个 Spec Requirement 8~12 条（不超过 15 条），单个 Spec <300 行（>700 行必须拆分）
Spec 格式	proposal / design / spec / tasks 四件套（OpenSpec），或 spec / plan / tasks（SpecKit），Requirement-Scenario（GIVEN/WHEN/THEN）结构
编写顺序	Specify → Plan → Tasks → Implement，每阶段经人工确认后进入下一阶段
提交时机	每个 Change 闭环提交：先提交 delta specs，AI 按 tasks 实现，归档时执行 Delta Spec 同步到主规格
人工确认	四道防线：架构师 → 产品 → AI 交叉检查 → 代码反验
质量门禁	满足 SDD 底线要求：有知识基线、编码前有 Spec、Spec 结构化、编码后有验证

推荐理由：

- 质效数据：产码可用率 80%~90%、功能遗漏率 2%~4%，质量显著提升；

- 降低对话要求：SDD 工具通过标准化命令与固定工作流（OpenSpec/SpecKit/BMad 的命令行、plan/tasks 等）驱动 AI 产码，人机交互以结构化 Spec 与 tasks 为准，降低了对员工与 AI 对话（提示词工程）能力的要求，普通开发人员即可上手，摆脱模式一对个人 AI 交互能力的依赖；
- 知识可追溯：需求→Spec→代码链路完整，人员流动不造成知识断层；
- 可并行协作：多 Change 并行、分工明确，团队协同效率高；
- 防偷懒防遗漏：结构化 tasks 逐项执行并逐条反验 Spec，减少 AI 偷懒与功能遗漏；
- 管理抓手：Spec 与代码结构可自动核查，便于组织级质效管控。

模式二对人员能力要求相对较低、具备规模化推广条件，是组织级 SDD 落地的**主力模式**。

3.4 模式三：Spec-As-Source（规范即源码）

定义：使用 SDD 工具将领域专业知识显性化为方法级结构化 Spec，**规范是主要工件，人只编辑规范**，代码完全由 AI 生成并通过测试验证，人尽量不直接修改代码。代码是 Spec 的派生产物，可随时从 Spec 重新生成。

工作方式：领域专家编写方法级 Spec → AI 严格按 Spec 生成代码 → 人通过测试用例验证输出正确性，全程无需人机对话纠偏。

适用项目特征/场景：

- 知识驱动型系统：领域规则密集、可完全显性化的专业领域（如电力调度、金融风控、医保结算规则等）；
- 高可靠性严肃业务：对代码质量与可审计性要求极高、需要最低 Bug 率的领域；
- 长期演进的领域核心资产：希望知识不依赖特定开发者、代码可随时从 Spec 重新生成的系统；
- 需深度领域专家参与、前期投入充足的重点项目。

Spec 提交要求：

维度	要求
架构设计文档	4A 架构设计文档(BA/AA/DA/TA) 先行完成并随方法级 Spec 一并提交，作为 Spec 的上游依据；架构变更时同步更新文档
Spec 粒度	方法级，领域规则、异常路径、边界条件完全显性化，Spec 完备度 ~100%

Spec 格式	结构化 Requirement-Scenario，业务规则细到可翻译为代码与测试的粒度
编写顺序	领域专家编写方法级 Spec → AI 严格按 Spec 生成代码 → 人通过测试用例验证
提交时机	所有变更必须通过修改 Spec 驱动，禁止直接修改代码；Spec 提交为主提交，验证通过后提交派生代码
人工确认	人机协同轮次趋近于 0；人聚焦于 Spec 编写质量与测试用例编写
质量门禁	Spec-Code 同步率 100%，代码可从 Spec 随时重新生成，不依赖特定开发者

推荐理由：

- **质量最高：**Spec 完备度~100%，产码可用率≈100%、功能遗漏率≈0%，人机协同轮次趋近于 0；
- **知识资产化：**领域知识完全显性化为结构化 Spec，不依赖特定开发者，代码可随时从 Spec 重新生成；
- **可审计性强：**需求→Spec→代码→测试全链路可追溯，适合高合规性要求的严肃业务领域；
- **长期复利：**一次投入编写方法级 Spec，后续迭代仅需修改 Spec 并重新生成代码，维护成本递减。

注意事项：前期投入较高（Spec 编写可占工作量 46%），ROI 不以代码行数度量，需要深度领域专家参与；建议知识驱动型重点项目优先考虑，一般项目可在模式二成熟运行后逐步深化至模式三。

3.5 工程结构索引要求（所有模式统一）

各项目技术栈、团队习惯差异较大，SDD 不强制统一目录结构模板，但要求每个 SDD 项目在项目仓库根目录 提供一份 SDD 索引文件(命名 SDD-INDEX.md)，作为项目知识基线的入口，供人、AI 与评审者快速定位各类资产与关键元数据。

索引文件必填内容：

类别	必填元数据
----	-------

项目元数据	SDD 策略（Spec 优先 / 逆向 Spec）、SDD 模式、SDD 工具（OpenSpec/SpecKit/BMad/无）及版本、代码生成模型、AI 编码工具、Spec 管理员（文档负责人）
需求文档	客户需求文档路径
架构文档	业务架构 BA、应用架构 AA、数据架构 DA、技术架构 TA 设计文档路径；架构决策记录 ADR 路径；详细设计文档与页面原型路径
项目宪法	constitution.md 或等价编码规范文件路径
Spec 资产	主规格目录、增量 Spec（delta specs）位置、活跃 Change 目录、归档目录、Spec 命名规范
代码资产	主源码目录（多子工程/前后端时逐项列出）、测试目录、生成代码目录、构建/发布配置
测试资产	测试用例目录、测试报告位置、Spec-Scenario→测试用例映射
知识资产	Skill 使用清单、编码规则文件、知识库目录、Prompt 模板、结项归档位置

SDD-INDEX.md 模板（以 OpenSpec 为例）：

注：本示例以 *OpenSpec* 为例展示索引文件的模板字段。*** 标注**必填属性**（即 *OpenSpec* / *SpecKit* / *BMad* 均具备的共性属性）；未标 *** 的为工具特有或可选项。若使用 *SpecKit* / *BMad*，可依据其目录与产物命名（如 *.specify/*、*spec/*、*project-context.md* 等）调整相应属性名称与路径。

```
# SDD 索引

## 项目元数据
- * 项目特征：<项目特征描述，说明选择 SDD 策略和 SDD 模式的原因>
- * SDD 策略：Spec 优先
- * SDD 模式：Spec-First
- * SDD 工具：OpenSpec v1.3.1
- * 代码生成模型：DeepSeek V4 Pro
```

东软秘密

- * AI 编码工具: VS Code + GitHub Copilot

- * Spec 管理员: 张 XX (架构师兼任)

需求文档

- * 客户需求: docs/requirements/

架构文档

- * 业务架构 BA: docs/architecture/业务架构 BA/

- * 应用架构 AA: docs/architecture/解决方案架构 SA/

- * 数据架构 DA: docs/architecture/解决方案架构 SA/

- * 技术架构 TA: docs/architecture/技术架构 TA.md

- * 规约: docs/standards/规约 Standard/

- * 编码规范: docs/standards/规约 Standard/Java 开发规范.md

- * 工程结构描述: README.md

- * 架构决策记录 ADR: docs/architecture/架构决策记录 ADR/

- 详细设计: docs/design/

- 页面原型: docs/prototype/

项目宪法

- * 知识基线/项目上下文: openspec/config.yaml

- * 项目宪法: openspec/specs/constitution.md

- 项目记忆: xxx/memory/

Spec 资产

- * (主) 规格目录: openspec/specs/

- * 设计资产: openspec/changes/<change-name>/ (目录下包含 design.md、plan.md、data-model.md 等)

- * 一致性验证记录: openspec/changes/<change-name>/check/

- 增量 Spec: openspec/changes/<change-name>/specs/

- 活跃 Change: openspec/changes/

- 归档: openspec/changes/archive/

- Spec 命名规范: spec-standard.md

代码资产

- * 源码: src/ (后端 java-service/、前端 web/)

```
## 测试资产
- * 单元测试用例: test/
- * 集成测试用例: test/
- 系统测试用例: test/
- 测试报告: reports/

## 知识资产
- * 知识库: docs/knowledge/
- * 编码规则: docs/coding-standards/rules/
- Skill: docs/skills/
- Agent: docs/agents/
- Prompt 模板: docs/prompts/
```

提交与维护要求:

- 索引文件在项目 SDD 启动时创建，随工程结构调整实时更新；
- 索引文件作为项目知识基线的组成部分，纳入月度巡检；

四、SDD 落地的组织建议

事业部在落地 SDD 时，不应依赖“每个项目各自摸索”，而应通过**分类接入、角色到位、试点撬动、数据闭环**实现标准化的规模化落地。

4.1 事业部落地 SDD 的总体思路

事业部落地 SDD 遵循“**角色到位、分类接入、试点撬动、数据闭环**”四步法：

1. **组织结构到位**：明确 SDD 负责人、教练、Spec 管理员等角色，落实 Spec 管理责任；
2. **盘点与分类**：盘点本事业部在管项目，按项目特征（新/存量、复杂度、生命周期、协同规模）进行分类并定级，为每类项目匹配 SDD 模式与 Token/算力资源配比；
3. **试点→拓面→自运转**：BU 先跑通 2~3 个核心试点，沉淀接入包与培训，再批量拓面，最终形成自运转与效能闭环；
4. **质效管控**：基于两单一务、研效平台建立人机协同效能及质量数据基线，通过量化数据驱动科学决策；

接入包说明：接入包指适用于当前 BU/行业/技术框架的通用性 SDD 落地套件，包括但不限于：工程结构模板、Spec 样例、架构设计方法、质量检查清单、培训课件等。由 BU SDD 教练在试点阶段沉淀并持续迭代。

4.2 组织结构与角色建议

角色	层级	职责	建议配置
事业部 SDD 负责人	事业部	SDD 整体规划、目标对齐、资源（Token/算力/人力）协调、月度对标	1 人（可由技术总监/研发负责人兼任）
BU SDD 教练	事业部	项目辅导、方法培训、问题解答、经验提炼与分享、接入包维护	每事业部 1~3 人（从试点骨干中选拔）
Spec 管理员（Spec Owner）	项目	Spec 体系管理 ：Spec 模板落地、Spec 质量门禁、Spec-Code 一致性巡检、归档管理、Spec 版本变更治理	每项目 1 人（架构师或资深开发兼任）
架构师	项目	4A 架构设计、Spec 分层展开、constitution 编写、架构决策维护、产码审查	每项目 1 人
PM/编排者	项目	任务拆分、Change 分配、人机协同编排、进度与质量门禁把控	每项目 1 人
开发工程师	项目	Spec 编写、AI 产码委托/补充、代码核查与验证、测试用例补充	按模块分工（一人一个或多个 Change）
测试工程师	项目	基于 Spec 的测试用例设计、自动化测试、质量评估	按模块分工

Spec 管理是 SDD 落地的关键角色：建议每个 SDD 项目明确 1 名 Spec 管理员，负责 Spec 的准入（格式/粒度/一致性）、变更（先改 Spec 后改代码）与归档（结项即沉淀），并作为 Spec-Code 一致性巡检的负责人。

4.3 项目特征与模式匹配

项目特征	推荐模式	推荐工具	说明
核心模块/知识密集型	模式三 （或模式二）	OpenSpec/ SpecKit	知识驱动型专业领域，质量优先

中长期产品/持续迭代/新开发通用/平台中台	模式二（主力）	OpenSpec/ SpecKit	规模化推广的主力层；平台项目须额外关注 API Spec 契约与向后兼容
存量重构/维护增强	逆向 Spec + 模式二	OpenSpec/ SpecKit	先逆向 Spec 建立基线，新功能 Spec 优先
数据/算法密集型	模式二	OpenSpec/ SpecKit	核心 Spec 为数据流水线与模型卡，通过 Change 管理持续迭代
系统集成/接口适配	模式二	OpenSpec/ SpecKit	以集成契约 Spec（接口协议/数据映射/SLA）为核心，业务规则 Spec 从简
短期/紧急/原型验证	模式一	无/轻量	快速交付优先，后期补课成本自担；
技术迁移/基础设施变更	模式一	无	最小 Spec 聚焦迁移范围与验证标准，不要求完整业务 Spec

4.4 落地路线图

第一步 试点（1~3月）	第二步 拓面（4~6月）	第三步 自运转（持续）
└ 确定2~3个核心试点项目	└ 批量接入（新建为主）	└ SDD项目覆盖率通报
└ 试点覆盖新开发+存量项目	└ 存量项目启动知识显性化	└ 效能数据反哺规范迭代
└ 输出接入包+培训课件	└ 全员分层培训	└ 结项即沉淀（Skill/资产）
└ 跑通SDD适配策略	└ 接入包迭代（方法包v2）	└ 能力从"推着走"到"自己能走"

4.5 模式设定及核验

基于公司两单一务系统、研发效能平台对项目采用的 SDD 模式进行设定及闭环核验，以客观数据驱动 SDD 的实质性落地。

模式设定

- **模式选择：**根据 SDD 模式选择策略，在两单一务中设置项目的 SDD 开发模式；
- **成本设置：**按项目特征（新/存量、复杂度、生命周期）、采用的 SDD 模式及预算初始化 AI 成本及 Token 额度；

- **持续度量：**持续采集项目里程碑进展、报工、成本、Token 消耗等过程数据，对 ROI 进行及时度量及风险预警；
- **月度通报：**以“SDD 项目覆盖率 + 深度（模式分布） + 效能（产码可用率/Token/投入产出） + 沉淀（Skill/知识库）”四个维度按月通报。

模式核验

根据项目组成员人机协同行为数据分析及项目配置库成果物检查结果，核验项目实际采用的 SDD 模式。核验规则详见《SDD 工程规范》的“达标证据清单”章节要求。

五、SDD 全流程的准备工作

SDD 涵盖从架构设计到代码归档的完整链路，不同阶段需要不同的准备工作：

5.1 团队准备

角色	职责	关键能力要求
架构师	主导 4A 架构设计、Spec 分层展开、工具链建设	4A 架构方法论、DDD 建模、Spec 编写
产品经理	需求确认、业务场景验收、原型评审	业务理解、场景描述精准度
开发工程师	Spec 编写、AI 产码核查、代码调整	Spec 编写、AI 代码审查、测试验证
测试工程师	测试用例设计、自动化测试、质量评估	测试策略、自动化测试工具

5.2 BA/SA 阶段准备

在进入 SDD 编码前，必须先完成业务架构设计（BA）和解决方案架构设计（SA），产出架构设计文档作为 SDD 的输入。

准备项	说明	参考资源
架构设计模板	4A 架构模板（BA 业务架构 + SA 解决方案架	neu_archi_knowledge_template/ 目录下的业务架构 BA、解决方案架构 SA 等模板。

	构），定义各阶段的产出物结构	业务架构的设计可使用 BA-document-generator Skill 辅助生成， BA 和 SA 文档的绘图可以分别使用 BA-drawing-mermaid-diagram 和 SA-drawing-mermaid-diagram 两个 Skill
AI 辅助模型	强推理模型适合 BA/SA 阶段的架构分析与设计	Claude Opus / GPT-5.4 / DeepSeek V4 Pro / Qwen3.7-Max
IDE 工具	选择团队最熟悉的 IDE，用于架构文档编写和后续 SDD 编码	VS Code + GitHub Copilot / Trae / Qoder / Claude Code

关键原则：架构设计是 SDD 的前置条件，没有经过 BA/SA 阶段产出的架构设计文档，SDD 工具无法生成高质量的 Spec 和代码。详细的架构设计流程参见 [新开发项目 SDD 实施指南](#) 和 [存量项目 SDD 实施指南](#)。

5.3 SDD 编码前准备

架构设计完成后、SDD 编码开始前，需要完成以下准备：

准备项	说明
SDD 工具	OpenSpec（v1.3.1+） / SpecKit（0.10+） / BMad Method，根据项目特征选择，详见各工具使用指南
代码生成模型	性价比模型适合产码阶段：DeepSeek V4 Pro / Qwen3.7-Max / GLM 5.2 / Sonnet

OpenSpec 与 SpecKit 工具对比（数据来源：GitHub 官方仓库及社区）：

对比维度	OpenSpec	SpecKit
定位	轻量、流畅、迭代式；面向存量+新建项目（brownfield + greenfield），个人到企业可伸缩	结构化、阶段门控的规范驱动流程；面向组织级标准化与企业约束
适用场景	存量项目改造、新建项目均可；强	0-to-1 新建项目、创意探索、迭

	调“先探索再提案”的流畅迭代，无需刚性阶段门	代增强（brownfield）；适合需要严格门控与合规审查的规范项目
设计产物详细程度	proposal + design + spec + tasks（四件套），层次分明；可随时更新任意产物	constitution → clarify → specify → plan → tasks → implement（七阶段）；附带 analyze/converge/checklist 辅助产物，产物链路更完整
学习曲线	低 — “easy not complex”，npm 一条命令安装，/opsx:propose 即可起步	中高 — 需先建立 constitution（项目原则），Python/uv 环境，多阶段命令需按序执行
AI 工具集成	30+ AI 助手（slash commands）	35+ AI 编码代理（specify integration list），含 CLI 和 IDE 两类
可自定义扩展便捷度	Community schemas(第三方 schema bundles)；可直接修改 artifact 文件，无层级体系	Extensions（新增能力）+ Presets（模板覆盖）+ Bundles（角色套装）+ overrides，四层优先级体系，扩展生态丰富
阶段门控	无刚性门 — “fluid not rigid”，可随时修改任意阶段产物	有阶段流程，可叠加 /speckit.analyze（交叉一致性检查）和 /speckit.checklist（质量清单）作为门禁
社区活跃度	64.3k ★ / 4.4k forks	125.9k ★ / 11.2k forks

官方互评：OpenSpec 认为 SpecKit “Thorough but heavyweight, rigid phase gates, lots of Markdown, Python setup — OpenSpec is lighter and lets you iterate freely”。两者均在 MIT 许可下开源，选择取决于项目对**流程刚性 vs 迭代灵活度**的偏好。

BMad Method 目前医疗事业部在用。

详见 [SDD 工具手册](#) 中的 OpenSpec 使用指南、SpecKit 使用指南与 BMAD 使用手册。

5.4 项目宪法准备（SA → SDD 的桥梁）

项目宪法（constitution.md）是 SA 阶段完成后、SDD 编码开始前的桥梁产物——它将架构设计决策翻译为 AI 可消费的技术约束规则。宪法的内容来源于 SA 阶段的架构决策，而非凭空编写：

宪法内容项	来源（SA 架构决策）
技术栈选型	SA 阶段确定的后端语言/框架、ORM、数据库、前端框架等
代码规范	SA 阶段定义的命名约定、禁止事项、分层规范
架构原则	SA 阶段确定的 DDD 分层、聚合根边界、依赖反转等
安全策略	SA 阶段规划的认证、授权、加密、防篡改方案
质量门禁	SA 阶段设定的测试覆盖率、代码静态扫描等标准

注：不同 SDD 工具中宪法的存放位置不同——OpenSpec 为 `openspec/specs/constitution.md`，SpecKit 为 `.specify/memory/constitution.md`，BMad 为 `project-context.md`。具体内容参见各工具使用指南。

六、SDD 实施通用原则

以下原则适用于所有类型的项目：

6.1 五连续统一原则

从需求到交付保持五个连续性，确保链路不断裂：

- 领域：**分类为业务架构、应用架构、数据架构、技术架构，架构要实现从业务到 IT 的连续统一。
- 范围：**分类为战略架构、分段架构、能力架构，架构要实现从宏观到微观的连续统一。
- 抽象层级：**分类为上下文级架构、概念级架构、逻辑级架构、物理级架构，架构要实现从逻辑到物理的连续统一。
- 状态：**分类为基线架构、目标架构，架构要实现从起点状态到目标状态的连续统一。
- 复用度：**分类为基础级架构、通用级架构、行业级架构、组织级架构，架构要实现从一般到特殊的连续统一。

6.2 Change 粒度控制

指标	建议范围	说明
单个 Change 任务数	8~12 个（最佳），上限 35 个	超过 40 个任务 AI 生成质量显著下降
单个 Change 代码量	200~400 行	过大 AI 跳过边缘逻辑，过小流程开销大
单个 Spec Requirement 数	8~12 条	过多导致 AI 上下文过载
单次 AI 执行	1 个 Change	避免跨 Change 上下文切换

6.3 人工确认是保障

每个阶段的产出必须经人工确认后才进入下一阶段：

- 架构设计 → 架构师确认架构决策
- Spec 编写 → 产品+架构师确认业务场景和规则
- Plan/Tasks → 开发确认技术方案和任务拆分
- 代码实现 → 开发核查 AI 产码质量
- 测试 → 测试确认覆盖度和通过率

6.4 Spec 管理与同步

- 所有变更先更新 Spec，再修改代码
- 人工修改代码后必须回写 Spec
- Code Review 同步检查 Spec 变更
- 定期执行 Spec-Code 一致性巡检

七、常见问题与应对策略

7.1 AI 幻觉

表现：AI 虚构不存在的功能和 API、字段名不一致、编造不存在的业务信息

应对：

东软秘密

- Spec 作为“基点”，所有代码必须与 Spec 逐条匹配
- 自动化测试验证核心场景
- 静态分析辅助（PMD/P3C 等工具自动检查）
- 人工审查关键业务逻辑

7.2 AI 偷懒

表现：大单体不分层、共性不抽取、Spec 中已明确的约束仍被遗漏

应对：

- 任务粒度细化，单个任务聚焦一个文件
- Checklist 驱动，每个任务附带规范检查清单
- Spec 中显式声明测试要求、注释要求、异常处理要求
- 异常路径显式列出，在 Spec 中显式列出必须覆盖的异常路径和边界条件
- 边缘条件拆分为独立子任务，强制 AI 逐一处理

7.3 上下文过载/AI 记忆丢失

表现：Change 过大时 AI 约束遗漏、状态流转被简化

应对：

- 控制 Change 粒度（8~35 个任务）
- 分层加载上下文
- 每个任务附带规范检查清单

7.4 文档与代码不同步

表现：接口变化未同步更新文档，后续 AI 基于错误上下文产出

应对：

- 约束文件机制，强制 AI 代码变更时反写文档
- 归档时执行 Delta Spec 同步
- 定期执行 Spec-Code 对齐检查

八、效能提升关键因素

东软秘密

基于 9 个试点项目的实践数据，以下因素对效能提升影响最大：

因素	影响程度	说明
Spec 完备度	★ ★ ★ ★ ★	有 Spec 约束可用率 75%，无 Spec 降至 40%
Change 粒度	★ ★ ★ ★	过大 AI 偷懒，过小流程开销大
架构设计粒度	★ ★ ★ ★	细化到类/方法/规则，产码可用率显著提升，但要考虑项目的 ROI 要求
模型选择策略	★ ★ ★	分级模型策略有效降低 Token 成本
工具链成熟度	★ ★ ★	成熟的 Skill/Prompt 体系减少人工干预

九、子文件索引

文件	内容
新开发项目 SDD 实施指南	Spec 优先策略的完整实施指南，含 4A 架构设计、Spec 分层展开、AI 编码全流程
存量项目 SDD 实施指南	逆向 Spec 策略的实施指南，含 4A 架构补建、分模块逆向生成 Spec、维护增强
AI 使用经验总结	模型选择、Token 优化、人机协同、质量保障等 AI 使用经验
Spec 编写指南	Spec 结构、编写要点、粒度控制、质量检查、常见反模式与对策

参考资源：

- [架构知识模板](#)
- OpenSpec 使用指南：[SDD 工具手册/OpenSpec 使用指南.md](#)
- SpecKit 使用指南：[SDD 工具手册/SpecKit 使用指南.md](#)
- BMad 使用手册：[SDD 工具手册/BMAD 使用手册.md](#)

东软秘密

SDD 工程规范（试行）

东软集团股份有限公司

目录

- 一、总则 1
 - 1.1 目的 1
 - 1.2 核心原则 1
 - 1.3 术语定义 1
 - 1.4 SDD 模式与适用 2
- 二、强制条款（必须遵守） 2
 - 条款 1：项目知识基线必须存在 2
 - 条款 2：编码前必须有 Spec 3
 - 条款 3：Spec 必须符合结构格式 3
 - 条款 4：Spec 中间设计产物必须存在且先于代码 4
 - 条款 5：编码后必须有验证 5
- 三、推荐实践 5
 - 3.1 架构设计推荐 6
 - 3.2 Spec 管理推荐 6
 - 3.3 知识管理推荐 6
- 四、达标认定方式 7
 - 4.1 达标判定流程 7
 - 4.2 达标判定标准 7
 - 4.3 达标证据清单 7
- 五、附录 8
 - 附录一 规范与其他文件的关系 8
 - 附录二 SDD-INDEX.md 元数据说明 9

一、总则

1.1 目的

本规范是公司级 SDD（规格驱动开发）的**底线标准**，目的是确保 80%以上项目落地 SDD 时，具备**统一的质量下限和可判定的达标依据**。

本规范**不替代**具体的操作指南和工具手册，而是定义“**做了什么**”和“**最低做到什么程度**”。具体“**怎么做**”请参考 SDD 工程方法指南及对应的项目类型指南。

1.2 核心原则

	规范约束边界原则：	
	✔ 约束“有没有” — 有没有 Spec、有没有验证、有没有知识基线	
	✔ 约束“最低格式” — Spec 的结构格式、业务规则的表达格式	
	✘ 不约束“用什么做” — 不强制指定 SDD 工具和模型	
	✘ 不约束“做到多精细” — 不强制完整 4A、不强制 Spec-Code 完全同步	
	一句话：规范是底线，不是天花板。	

1.3 术语定义

术语	定义
Spec（规格）	AI 编码前的行为约束文档，描述系统“应该做什么”，是人与 AI 之间的契约
知识基线	项目级上下文文档，包含工程结构、技术栈、编码规范、项目知识等
Requirement	Spec 中的需求条目，用 MUST/MUST NOT/SHOULD 格式描述系统行为约束
Scenario	Spec 中的场景描述，用 GIVEN/WHEN/THEN 格式定义可验证的行为预期

SDD 工具	OpenSpec、SpecKit、BMad Method 等结构化 SDD 工具链
--------	---

1.4 SDD 模式与适用

SDD 三种开发模式：

- 1. 模式一（基于架构设计的对话式 AI Coding）：架构师编写架构设计文档，开发人员通过对话方式使用 AI 编码，产码后逆向更新设计文档，不使用 SDD 工具。
- 2. 模式二（Spec-First）：使用 SDD 工具（OpenSpec/SpecKit/BMad 等）结构化管理架构设计与规范，先写 Spec 再产码，Spec 与代码同步演进。
- 3. 模式三（Spec-As-Source，规范即源码）：Spec 精细到方法级，人只编辑规范，代码完全由 AI 生成并通过测试验证。

模式一仅适用于短期紧急交付、原型验证、Bug 修复、技术迁移等兜底场景，常规项目应采用模式二或模式三。

三种模式的详细说明、适用场景、工具选择和操作方法见《SDD 工程方法指南》。

SDD 模式	规范适用级别	说明
模式一（基于架构设计的对话式 AI Coding）	全部强制条款（条款 3 按模式差异化判定）	以“逆向更新的设计文档”作为条款 2/5 的 Spec 依据；条款 3 以设计文档中显性化的核心业务规则、接口协议与关键约束（可执行约束）作为最低要求，不强制 GIVEN/WHEN/THEN 格式
模式二（Spec-First）	全部强制条款	主力模式，按条款逐项判定
模式三（Spec-As-Source）	全部强制条款	规范即源码，天然满足条款 3

二、强制条款（必须遵守）

以下条款为 SDD 达标的必要条件，任何一条不满足即不被认定为“已落地 SDD”。各条款的最低要求按 SDD 模式差异化判定（模式一以逆向更新的设计文档作为 Spec 依据，见 1.4 与条款 3 的模式对应表），并非所有项目都按同一格式执行。

条款 1：项目知识基线必须存在

东软秘密

要求：项目必须有知识基线文档，使人与 AI 均能理解项目上下文。

检查项	最低要求
SDD 索引文件	项目根目录存在 SDD-INDEX.md，元数据（SDD 策略/模式/工具等）齐全
基线文件/项目上下文	• OpenSpec: openspec/config.yaml + constitution.md • SpecKit: .specify/memory/constitution.md + init-options.json; • BMad: project-context.md • 模式一：存在同等/类似内容的说明文档
工程结构描述	包含项目的技术栈、模块划分、目录结构
编码规范	至少有一份编码规范文件（内容可因地制宜）
人机友好	所有知识文档要层次分明、路径明确，对人类阅读友好、AI 可直接解析，如 Markdown 等

验证方式：项目根目录存在 SDD-INDEX.md，文件必填字段完整、引用的文件路径可访问，且内容准确无误。

条款 2：编码前必须有 Spec

要求：每次核心开发任务启动前，必须先完成 Spec 编写。

检查项	最低要求
Spec 文件存在	• 有需求 Spec 描述文档 • 模式一：可以是完整 Spec 或融入设计文档的轻量 Spec
时间顺序正确	Spec 创建/更新时间 等于/早于 对应代码的提交时间

验证方式：检查 Spec 文件存在性，以及与代码的时间戳对比。

条款 3：Spec 必须符合结构格式

东软秘密

要求：Spec 必须包含结构化内容，不能是自由文本描述。

检查项	最低要求
含 Requirement	至少包含 1 条 Requirement，使用 MUST/MUST NOT/SHOULD 格式
含 Scenario	每条 Requirement 至少有 1 个 Scenario，使用 GIVEN/WHEN/THEN 格式（模式一：不强制该格式）
粒度受控	单个 Spec 不超过 700 行，Requirement 不超过 15 条
业务规则可执行	规则描述是具体的、可转化为代码或测试的约束，不是模糊的描述性文字

模式一与各工具产物的对应（如何满足条款 3）：

场景/工具	满足条款 3 的方式
模式一（设计文档作为 Spec）	不强制 GIVEN/WHEN/THEN 格式；核心业务规则、接口协议、关键约束须在设计文档中显性化为可执行约束（MUST/SHOULD 或规则列表），非模糊描述性文字
OpenSpec	proposal/spec/design/tasks，spec 含 Requirement-Scenario 结构
SpecKit	.specify/ 下 requirements/scenarios，GIVEN/WHEN/THEN 结构
BMad Method	spec-*.md/story-*.md 的“验收标准”建议采用 GIVEN/WHEN/THEN 结构；列表式验收标准须将核心规则显性化为可执行约束

验证方式：检查 Spec 文件内容是否包含 Requirement 结构和 Scenario 结构（模式一按上表从宽判定）。

条款 4：Spec 中间设计产物必须存在且先于代码

要求：每次核心开发任务启动前，须先完成 Spec 的中间设计产物（设计方案、任务拆分等），

东软秘密

且提交时间早于对应代码。

检查项	最低要求
设计产物存在	• OpenSpec proposal/design/plan/tasks • SpecKit spec/plan/tasks • BMad spec/story • 模式一：详细设计文档
时间顺序正确	设计产物提交时间早于对应代码的提交时间
内容完整	设计产物含技术方案、任务拆分等结构化内容（模式一按差异化判定，以设计文档中的详细设计为准）

验证方式：检查 Git 提交中设计产物的存在性 + 提交时间戳对比。

条款 5：编码后必须有验证

要求：编码完成后必须执行验证动作，须覆盖从业务到代码的连续一致性，验证结果必须记录。

检查项	最低要求
验证动作已执行	有自动化验证报告或人工核查记录
验证覆盖 Spec	验证内容覆盖了 Spec 中的核心 Requirement
验证结果已记录	有验证报告文件或在代码提交中附带验证说明
业务-架构-Spec-代码的连续一致性	验证记录覆盖从客户需求→架构设计→Spec→中间设计产物→代码→测试的全链路一致性核查结论

验证方式：检查验证报告文件/验证日志/工具输出记录，含业务与代码一致性核查记录。

三、推荐实践

以下实践是提升 SDD 落地质量的**建议**，鼓励项目团队按需采用，但不作为达标认定的硬性条件。

东软秘密

3.1 架构设计推荐

架构设计方法遵循公司《架构治理-架构设计方法参考》，架构设计内容参考模板 neu_archi_knowledge_template_v0.6，具体设计深度根据项目情况而定。

推荐	说明	不强制的原因
遵循《 架构治理-架构设计方法参考 》	公司统一的架构设计方法， 点击查看培训视频	根据项目情况而定
参考 neu_archi_knowledge_template_v0.6	架构设计内容模板（BA/AA/DA/TA、ADR 等）	按项目情况裁剪使用

3.2 Spec 管理推荐

推荐	说明	不强制的原因
Spec 粒度受控	单个 Spec 8~12 条 Requirement（不超过 15 条）、<300 行，>700 行必须拆分	小型变更可适当放宽
Spec 归档结构统一	按标准目录结构（.sdd/specs/knowledge 等）统一归档管理	目录结构是工具层面约定，不影响本质
Spec 内容准确率	业务规则可执行、与需求/架构一致，不含 AI 过度推理内容	需人工确认，投入成本较高
每条 Requirement 至少 2 个 Scenario	1 正常+1 异常	简单变更 1 个 Scenario 可能足够

3.3 知识管理推荐

推荐	说明	不强制的原因
知识库按标准结构	业务架构 BA/解决方案架构 SA/架构决	不同项目知识分类可能不

组织	策记录 ADR/规格 Spec/规约 Standard	同
沉淀项目级 Skill	将项目特有流程编码为 Skill	需要较高 Skill 开发能力
发布 Skill 到 Skill 市场	全公司复用	视项目是否有可复用的 Skill

四、达标认定方式

4.1 达标判定流程



4.2 达标判定标准

项目被认定为“已落地 SDD”（达标），须**满足全部 5 条强制条款**；推荐实践为加分项，不作为达标门槛。

4.3 达标证据清单

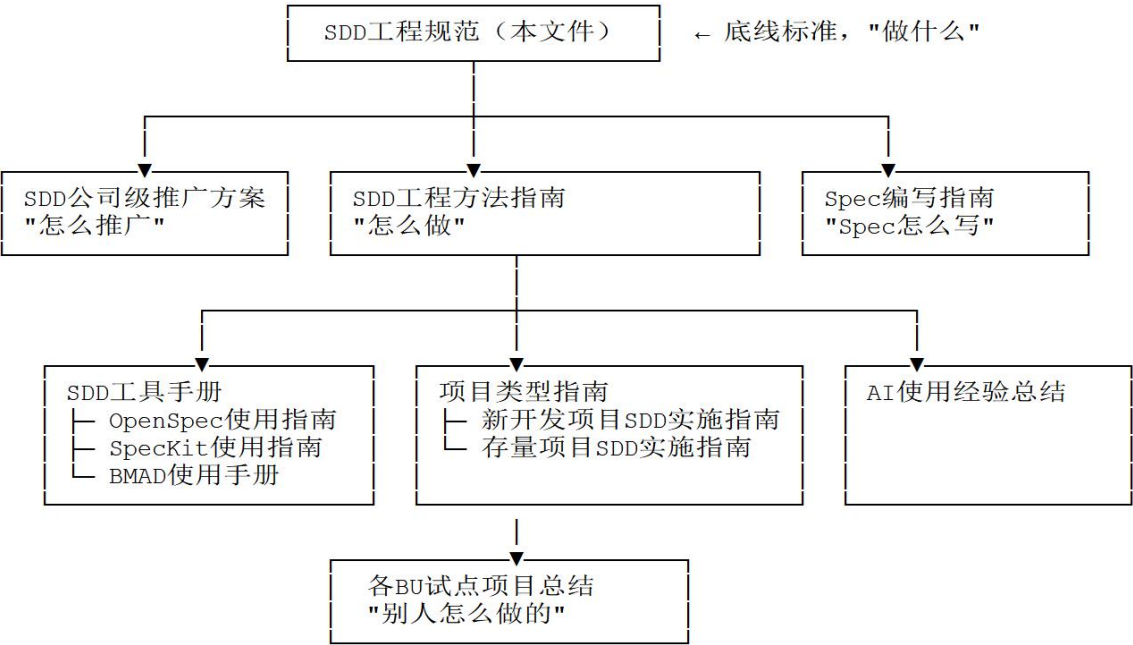
项目需提供以下**最低证据**证明已落地 SDD（与 5 条强制条款一一对应），所有证据均可通过项目根目录 `SDD-INDEX.md` 索引文件核查（索引指向知识基线、编码规范、Spec、测试与知识资产等路径）：

对应条款	证据类型	具体要求
条款 1	知识基线	项目根目录 <code>SDD-INDEX.md</code> 元数据齐全； <code>onfig.yaml/constitution.md/project-context.md</code>

		（BMad）等基线文件存在； 含工程结构描述与至少 1 份编码规范； 文档对人类阅读友好、AI 可解析；
条款 2	Spec 先于代码	有需求 Spec 文档，Spec 创建时间早于对应代码提交时间；
条款 3	Spec 结构合规	所有 Spec 均满足结构格式要求（含 Requirement/Scenario、粒度受控、业务规则可执行； 模式一按差异化判定，以设计文档中显性化的可执行约束为准）；
条款 4	中间设计产物	每个 Spec 均含中间设计产物（design/plan/tasks 或设计文档），提交时间早于对应代码，内容含技术方案与任务拆分；
条款 5	验证记录	每份 Spec 均包含验证日志，覆盖 Spec 核心 Requirement 并包含从业务（客户需求）→架构→Spec→中间设计产物→代码→测试的连续一致性核查记录；

五、附录

附录一 规范与其他文件的关系



附录二 SDD-INDEX.md 元数据说明

SDD-INDEX.md 是项目知识基线的统一入口，各属性的字段样例见《SDD 工程方法指南》3.5 的 SDD-INDEX.md 模板（以 OpenSpec 为例，SpecKit/BMad 按其目录与产物命名调整）。

类别	属性	必填	属性说明
项目元数据	项目特征	✔	项目特征描述，说明选择 SDD 策略与 SDD 模式的原因
	SDD 策略	✔	Spec 优先 / 逆向 Spec
	SDD 模式	✔	基于架构设计的对话式 AI Coding / Spec-First / Spec-As-Source（见 1.4）
	SDD 工具及版本	✔	OpenSpec / SpecKit / BMad / 无
	代码生成模型	✔	产码使用的 AI 模型
	AI 编码工具	✔	IDE 与编码助手
	Spec 管理员	✔	文档负责人（架构师或资深开发兼任）

东软秘密

需求文档	客户需求	✓	客户需求文档路径
架构文档	业务架构 BA	✓	业务架构设计文档路径
	应用架构 AA	✓	应用架构设计文档路径
	数据架构 DA	✓	数据架构设计文档路径
	技术架构 TA	✓	技术架构设计文档路径
	规约 Standard	✓	规约/标准文档路径
	编码规范	✓	项目的编码规范文档
	工程结构描述	✓	项目的技术栈、模块划分、目录结构等说明
	架构决策记录 ADR	✓	架构决策记录路径
	详细设计	✗	详细设计文档路径
	页面原型	✗	页面原型路径
项目宪法	知识基线/项目 上下文	✓	知识基线配置文件（config.yaml / project-context.md 等）
	项目宪法	✓	constitution.md 或等价编码规范文件
	项目记忆	✗	项目记忆目录（如 .specify/memory/）
Spec 资产	主规格目录	✓	主规格 Spec 目录
	设计资产	✓	Spec 对应的设计文档 （design/plan/data-model/tasks 等）
	一致性验证记 录	✓	每份 Spec 与代码和测试用例的一致性验证 记录位置

	增量 Spec	✖	增量 Spec (delta specs) 位置
	活跃 Change	✖	进行中的 Change 目录
	归档	✖	已完成 Change 归档目录
	Spec 命名规范	✖	Spec 命名规范文件
代码资产	源码	✔	主源码目录 (多子工程/前后端逐项列出)
测试资产	单元测试用例	✔	单元测试目录
	集成测试用例	✔	集成测试目录
	系统测试用例	✖	系统测试目录
	测试报告	✖	测试报告位置
知识资产	知识库	✔	项目知识库目录
	编码规则	✔	编码规则文件目录
	Skill	✖	项目 Skill 清单
	Agent	✖	智能体定义
	Prompt 模板	✖	提示词模板目录

说明:

- ✔ 为必填属性, ✖ 为可选项; 不同工具的目录与产物命名可相应调整。
- 属性值为多条数据时, 可用英文逗号(,)分割, 如: docs/standards/规约 Standard/Java 开发规范.md, README.md

核心理念: 本规范是 SDD 推广的**底线**, 不是天花板。它确保每个项目做到“有 Spec、有验证、有积累”这三件事, 至于用什么工具、做到多精细, 由项目团队根据自身情况决定。

东软秘密

SDD 公司级推广实施方案

东软集团股份有限公司

目录

一、推广项目范围 1

 1.1 确定依据 1

 1.2 按项目类别划分 1

 1.3 时间边界 2

 1.4 存量项目纳入 2

 1.5 推广实施的 BU 范围 2

二、推广计划 2

 2.1 各 BU 提交推广计划 2

三、SDD 成果物审查 3

 3.1 审查机制 3

 3.2 成果物清单 3

 3.3 成果物上 Git 要求 3

 3.4 审查频次 3

四、SDD 落地判定标准 4

 4.1 项目“已落地 SDD”的判定条件 4

 4.2 覆盖率口径 4

五、治理与考核机制 4

 5.1 组织保障 4

 5.2 考核与评估 5

六、其他要求 5

 6.1 项目 PM 负责制 5

 6.2 统一工具与数据采集 5

 6.3 两单一务系统 8

七、特殊情况说明 8

 7.1 豁免申请流程 8

八、附则 9

东软秘密

一、推广项目范围

1.1 确定依据

1. 以公司《项目分类分级标准》文档为统一口径，按**项目类别**确定是否纳入 SDD 推广范围；
2. **核心判定原则**：凡涉及我司软件资产的代码开发或修改的项目，均纳入 SDD 推广范围。

1.2 按项目类别划分

判定两步法：

1. 按《项目分类分级标准》确认项目类别与子类；
2. 判定是否涉及我司软件资产的代码开发或修改（存量项目含“有需求变更需修改代码”）；

项目类别	是否纳入 SDD	说明
研发项目（N-B1 新产品研发/产品换代升级/衍生品研发 B2/产品改进/共性技术研发）	✓ 纳入	全部子类均涉及软件开发
解决方案类—软件开发项目	✓ 纳入	核心推广对象
解决方案类—数据服务项目	✗ 不纳入	以数据治理/分析为主，无我方软件代码开发（含专项开发另行判定）
解决方案类—软件集成项目	✓ 纳入	涉及系统集成与接口开发
解决方案类—咨询规划设计项目	✗ 不纳入	产出为方案/设计，需提交设计方案到知识库
解决方案类—解决方案运维服务	✗ 不纳入	无代码开发（含专项开发另行判定）
系统集成类—软硬件集成	✓ 纳入	含软件开发部分则纳入
系统集成类—总集项目/产品供货	✗ 不纳入	无我方软件开发
自有产品类—产品定制开发	✓ 纳入	基于产品的二次开发
自有产品类—标准产品交付/产品运维服务/产品衍生	✗ 不纳入	无定制开发

东软秘密

工程服务类（固定总价外包 FP/项目制外包/T&M 人员外包）	✗ 不纳入	以人员/外包交付为主，无我方自主软件代码开发（含专项开发另行判定）
运营类（资产运营/服务运营）	✗ 不纳入	无软件开发
其他类（科研项目/内部管理项目）	✗ 不纳入	无软件交付开发（含专项研发另行判定）

1.3 时间边界

1. 新立项项目：自发文之日起新立项、新签订合同的项目，一律采用 SDD；
2. 在行项目：发文前已立项、仍在执行的项目，按“1.4 存量项目纳入”要求执行。

1.4 存量项目纳入

1. 尚未进入设计阶段的项目，自发文之日起一律采用 SDD；
2. 在行项目凡有需求变更、需要修改代码的，一律纳入 SDD 范围，需先完成逆向知识工程，新需求/新功能采用 SDD 方法；
3. 处于收尾阶段、无代码改动需求（仅运维/纯维护）的项目，暂不纳入，后续新需求再纳入执行。

1.5 推广实施的 BU 范围

- 各 BU 内，凡符合 1.2“纳入”标准的项目均纳入本次推广范围；BU 内无符合条件项目的，可不纳入本次推广；
- 各 BU 按 1.2 初步判定本 BU 纳入项目范围，随推广计划提交初步项目清单；在行项目梳理完成后，于 2026 年 9 月 10 日前提交确认版纳入项目清单。

二、推广计划

2.1 各 BU 提交推广计划

- 各部门于 2026 年 8 月 25 日 前向公司项目管理部提交本 BU 推广计划；
- 推广计划必须明确以下内容：
 - a. BU 级 SDD 负责人：明确 1 人（可由技术总监/研发负责人兼任）对本 BU SDD 推广负总责；
 - b. 在行项目梳理完成时间：不晚于 2026 年 9 月 10 日，完成本 BU 在行项目的盘点、分类与定级，提交确认版纳入项目清单；
 - c. 确认参与逆向知识工程试点的软件资产：每部门确认参与逆向知识工程试点的核

东软秘密

心软件资产（至少一个），明确其代码规模，已具备的设计文档等；

c. **逆向知识工程完成时间：**参与逆向知识工程试点的软件资产不晚于 2026 年 10 月 31 日完成知识显性化示范；其余纳入项目的完成时间由 BU 明确，不晚于覆盖率考核节点；

三、SDD 成果物审查

3.1 审查机制

- 公司 SDD 推进组对各项目成果物进行**不定期抽查审查**，审查结果纳入月度通报与考核。

3.2 成果物清单

审查项	审查内容	合格标准
架构设计文档	业务架构(BA)、应用架构(AA)、数据架构(DA)和技术架构(TA)	文档存在、内容完整、已提交公司 Git 仓库
SDD-INDEX.md	SDD 信息索引文件，以及对应的文件内容	项目根目录存在索引文件，且文件内容完整
Spec 文档	结构合规、粒度受控	含 Requirement/Scenario 结构化内容，以及业务规则，且行数在合理范围
中间设计产物	核心开发任务前有 design/plan/tasks 或设计文档	文件存在，内容完整，且提交时间早于对应代码（存量项目不校验时间）
验证记录	Spec 验证日志，覆盖 Spec 核心 Requirement 并包含从业务（客户需求）→架构→Spec→中间设计产物→代码→测试的连续一致性核查记录	每份 Spec 含验证日志，覆盖核心 Requirement

3.3 成果物上 Git 要求

- 架构设计文档、Spec 文档必须上传公司 Git 仓库；
- 具体要求按《SDD 工程规范（试行）》执行；

3.4 审查频次

东软秘密

- 不定期抽查：公司 SDD 推进组按需抽查，覆盖各 BU；
- 月度例行：结合 Git 自动化核查与月度对标，检查成果物上 Git 情况；

四、SDD 落地判定标准

正式达标认定依据详见《SDD 工程规范（试行）》，以下为概要。

4.1 项目“已落地 SDD”的判定条件

一个项目被认定为“已落地 SDD”（达标），须满足全部 5 条强制条款（详见《SDD 工程规范（试行）》），概要如下：

对应条款	判定项	最低要求
条款 1	项目知识基线	项目根目录存在 SDD-INDEX.md；基线文件（config.yaml/constitution.md/project-context.md 等）存在；含工程结构描述与至少 1 份编码规范
条款 2	Spec 先于代码	有需求 Spec 文档，Spec 创建时间早于对应代码提交时间
条款 3	Spec 结构合规	所有 Spec 满足结构格式（Requirement/Scenario、粒度受控、业务规则可执行；模式一按差异化判定）
条款 4	中间设计产物	每个 Change 含中间设计产物（design/plan/tasks 或设计文档），提交时间早于对应代码
条款 5	编码后验证	每份 Spec 含验证日志，覆盖 Spec 核心 Requirement，含 Spec 与代码和测试用例的一致性核查记录

4.2 覆盖率口径

- 公司级 SDD 覆盖率 = 已落地 SDD 项目数 ÷ 推广范围内项目数；
- 覆盖率目标：2027 年 1 月底达到 80%。

五、治理与考核机制

5.1 组织保障

责任主体	层级	职责
SDD 推进组（PISA）	公司级	标准制定、工具链维护、培训组织、成果物审查、

东软秘密

		Token 资源保障
考核评估组（项目管理部）	公司级	两单一务升级改造，SDD 考核与评估，提供统一采集工具进行效能数据采集，发布月度评估报告
BU SDD 负责人	事业部	BU 内 SDD 推广第一责任人，确保覆盖率目标达成、资源协调、月度对标
BU SDD 教练	事业部	项目辅导、达标复核、问题解答、经验提炼与分享
项目 PM	项目级	SDD 过程、成果物、质量负总责（详见六、其他要求）

5.2 考核与评估

考核维度	考核内容	计算方式	数据来源
SDD 覆盖率	推广范围内项目达标比例	通过 5 条强制条款的项目数 ÷ 推广范围内项目数	两单一务 + Git 自动核查
SDD 成果物合规率	5 条强制条款通过情况	5 条全部通过项目数 ÷ 已启动 SDD 项目数	Git 自动核查
知识沉淀活跃度	知识资产随代码同步更新情况	Spec-Code 同步率（代码变更伴随 Spec 更新的提交数 ÷ 总代码变更提交数）、可复用 Skill/Agent 数	Git + Skill 市场

考核结果运用：

- 月度通报：各 BU 考核指标排名
- 年度考核：SDD 覆盖率目标达成情况纳入各 BU 年度考核

六、其他要求

6.1 项目 PM 负责制

- 各项目 PM 对 SDD 的过程、成果物、质量负总责，确保项目按 SDD 规范执行、成果物按时上 Git、过程数据按时上报。

6.2 统一工具与数据采集

统一软件开发工具与数据采集工具，确保过程数据可比、可采集、可考核。

6.2.1 统一工具清单

工具类别	统一工具	说明
开发工具	VS Code + GitHub Copilot（或公司统一选定的 IDE/编码助手）	各 BU 统一，便于经验复用
SDD 工具	OpenSpec / SpecKit / BMad Method	OpenSpec/SpecKit/BMad 等，由项目根据 SDD 模式选配，详见公司 SDD 知识库
代码托管	公司统一的 Git 仓库平台	架构文档、Spec、代码统一入库
数据采集	两单一务 + Git 自动化核查 + 统一采集工具	由项目管理部提供

6.2.2 数据采集口径（核心指标）

第一类：推广覆盖

指标	计算方式	数据来源	采集频率	决策价值
SDD 覆盖率	通过 5 条强制条款的项目数 ÷ 推广范围内项目数	两单一务 + Git 自动核查	月度	衡量推广进度，80% 目标达成情况
SDD 成果物合规率	5 条全部通过项目数 ÷ 已启动 SDD 项目数	Git 自动核查	月度	区分“真 SDD”和“挂名 SDD”
各条款通过率	单条通过项目数 ÷ 已启动 SDD 项目数（条款 1~5 分别统计）	Git 自动核查	月度	定位薄弱环节，指导改进方向

第二类：AI 效能

指标	计算方式	数据来源	采集频率	决策价值
AI 架构设计采纳率	AI 生成的架构设计内容被采纳并保留在	数据采集工具	月度	衡量 AI 在架构设计阶段的产出质量，支撑架构设计

东软秘密

	最终文档中的比例			AI 工具 / 模型选型，指导架构师 AI 使用培训
AI 代码采用率	AI 建议被接受的代码行数 ÷ AI 建议总代码行数	数据采集工具	月度	衡量 AI 建议质量，可按模型对比，支撑模型选型决策

第三类：人机协同能力

指标	计算方式	数据来源	采集频率	决策价值
单个 Spec 平均 AI 对话轮次	从 Spec 创建到对应代码完成并验证通过，交付一个 Spec 过程中人与 AI 的平均对话轮次	数据采集工具	月度	综合反映 Spec 完备度和人机交互有效性：Spec 越完备、人员与 AI 交互能力越强，轮次越少；轮次持续偏高提示 Spec 质量不足或人员交互能力弱，可按 BU / 项目 / 人员维度对比，指导培训重点

第四类：知识沉淀

指标	计算方式	数据来源	采集频率	决策价值
Spec-Code 同步率	代码变更伴随 Spec 更新的提交数 ÷ 总代码变更提交数	Git 提交关联分析	月度	衡量知识资产是否随代码“活”着，防止 Spec 写完就过期
知识基线完整率	SDD-INDEX.md 必填字段完整的项目数 ÷ SDD 项目总数	Git 自动核查	月度	衡量项目知识显性化基础质量
可复用资产新增数	本期新增 Skill/Agent 数	Skill 市场	月度	衡量从经验到组织资产的转化

第五类：成本效率

指标	计算方式	数据来源	采集频率	决策价值
----	------	------	------	------

东软秘密

Token 产出效率	AI 生成且留存的代码行数 ÷ Token 消耗量	数据采集工具	月度	衡量 Token 投入的代码产出，可按模型/模式/BU 对比，支撑 Token 配额和模型选型决策
单位任务 Token 成本	Token 消耗金额 ÷ 交付任务数	数据采集工具+两单一务	月度	衡量 AI 编码的单位成本，评估 SDD 投入产出比
各阶段 Token 分布	架构设计、Spec 编写、代码生成、测试等阶段 Token 消耗	数据采集工具	月度	识别 Token 消耗结构是否合理，指导资源投入优化

6.3 两单一务系统

6.3.1 项目类别选项

- 两单一务系统新增“项目类别”字段，选项与《项目分类分级标准》一致；
- 同步标记**是否纳入SDD及匹配模式**，作为SDD项目识别、统计与月度对标的统一入口。

6.3.2 Token 配额分配规则

- 提前申请：**项目立项/启动时按需向公司申请 AI Token 用量；
- 分配维度：**结合项目合同额、预估利润、项目复杂度与开发工作量分配配额；
- 配额管理：**公司统一管理、月度核算；超额需追加申请；
- 申请与核算通过两单一务统一承载。

七、特殊情况说明

- 推广范围内的项目**直接纳入 SDD**（范围确定即纳入，无需另行申请加入）；
- 确因特殊情况无法纳入或无法执行的，按 **豁免申请流程** 申请，经批准后可不纳入。

7.1 豁免申请流程

适用范围：推广范围内因特殊情况无法纳入或无法执行 SDD 的项目。

申请流程：

项目 PM 提出申请（说明原因、豁免范围与期限）

→ BU SDD 负责人审核

东软秘密

- SDD 推进组审批
- 备案（月度汇总）

注意：豁免的项目在项目结束后需补充知识资产。

数量上限：豁免项目累计不超过推广范围内项目数的 **10%**，超出部分需提交公司审批。

八、附则

本方案未尽事宜及 SDD 具体操作方法，见公司 [SDD](#) 知识库，由产品创新与软件资产管理部持续更新维护。